

TP4 : Un Mud en TDD

Qualité de développement

BTS2 - SLAM

2026-2027

Résumé

Cette feuille vous permet de vous entraîner au développement dirigé par les tests.

Dans ce TP, vous allez mettre en application ce que vous avez appris dans le précédent TP, et vous entraîner au développement dirigé par les tests sur une nouvelle petite bibliothèque.

1 Récupérez le projet

Comme aux TP précédents, tout ce dont vous avez besoin — l'outillage de test et l'énoncé que vous êtes en train de lire — se trouve dans un dépôt sur GitHub :

<https://github.com/roza/tdd-backlog-mud>

Fabriquez-vous-en une copie personnelle : bouton vert **Use this template**, puis **Create a new repository**. Nommez votre dépôt `tdd-backlog-mud`, cochez **Private**, et validez. Ajoutez-moi ensuite en collaborateur (**Settings**, **Collaborators**, **Add people**, compte `roza`), sans quoi je ne pourrai pas relever votre travail.

Clonez enfin votre dépôt (remplacez `VOTRE-COMPTE` par votre identifiant GitHub) :

```
$ git clone https://github.com/VOTRE-COMPTE/tdd-backlog-mud.git
$ cd tdd-backlog-mud
$ ls
junit-platform-console-standalone-6.1.3.jar  tdd-backlog-mud.code-workspace
LISEZMOI.md                                TP4.pdf
```

Votre dépôt local est déjà relié au dépôt distant (appelé `origin`) et possède une branche `main` avec un commit : il n'y a rien d'autre à configurer.

Attention



- Ne faites **pas** de `git init` en plus du `git clone` : vous obtiendriez un dépôt dans un dépôt.
- Depuis les machines du Lycée, il faut un token valide (ou un autre moyen de connexion) pour accéder à votre compte GitHub.

1.1 Passer à JUnit 6

Dans les TP précédents, vous avez écrit vos tests avec JUnit 4, qui n'évolue plus depuis 2021. Dans ce TP, nous passons à la version actuelle, **JUnit 6** (vous l'avez peut-être déjà découverte dans le bonus du TP2). Les idées sont les mêmes : des méthodes annotées avec `@Test`, des assertions.

Tout tient dans **un seul** fichier jar (plus besoin de hamcrest), déjà présent dans votre dépôt. Pour éviter de retaper son long nom, on le range dans une variable du shell :

```
$ J=junit-platform-console-standalone-6.1.3.jar
```

À retenir

Avec JUnit 6, les imports deviennent :

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
```

On compile, puis on lance les tests avec :

```
$ javac -cp .:$J BoxTest.java
$ java -jar $J execute --class-path . --scan-class-path
```



Le sous-mot `execute` est obligatoire, et `--scan-class-path` demande à JUnit de trouver tout seul les classes de test.

Attention



- JUnit 6 exige Java 17 au minimum : vérifiez avec `java -version`.
- La variable `J` n'existe que dans le terminal où vous l'avez définie : si vous ouvrez un nouveau terminal, redéfinissez-la.

2 Rappel de la méthodologie

Nous vous proposons un *backlog* d'items à réaliser.

Rappel



Un **backlog** est une liste de tâches à réaliser. On associe aux tâches un **niveau de priorité**.

À retenir

Vous **devez** suivre la méthodologie suivante :

- prendre un élément du backlog
- le traduire par un test
- faire tourner les tests pour vérifier que le nouveau test échoue par manque de fonctionnalité (et non pas à cause d'une erreur de syntaxe)
- commiter le test
- implémenter la fonctionnalité
- faire tourner les tests pour vérifier que le test passe
- commiter cette nouvelle fonctionnalité
- passer à l'item suivant du backlog



3 Une bibliothèque pourquoi faire ?

Ce TP commence à vous préparer à la mise en place d'un MUD (Multi-user dungeon https://fr.wikipedia.org/wiki/Multi-user_dungeon). Dans un MUD il y a des lieux connectés par des passages, il y a des portes qui peuvent être verrouillées, il y a des coffres qui peuvent contenir des objets, il peut même y avoir des ennemis contre lesquels il faut se battre, etc...

La question importante est : comment peut-on représenter toute ces choses ? Nous allons les représenter par des objets ayant des attributs et des méthodes.

Afin d'organiser un peu notre travail, nous allons nous fixer un certain nombre de *features*. Ce terme provient de Git Workflow qui est un ensemble de bonnes pratiques à suivre lorsqu'on utilise Git.

À retenir

Pour faire simple, une feature peut représenter différentes choses comme par exemple :

- résoudre un bug
- développer une nouvelle fonctionnalité
- ajouter de la documentation
- ...

Une feature doit rester quelque chose de précis et d'assez "petit". Une liste de features est appelée un backlog. Nous allons donc nous consacrer maintenant à la feature : *Bibliothèque Box*. Chaque feature doit être développée dans une branche séparée.



4 Bibliothèque Box : Premiers éléments du backlog

Voici les deux premiers éléments du backlog :

- on veut pouvoir créer des boîtes
- on veut pouvoir mettre des trucs dedans

et nous allons les réaliser ensemble.

Question 1 On veut pouvoir créer des boîtes! Traduisez d'abord cet item de backlog en un test.

Nous devons traduire cet item en un test. Pour cela, avec un éditeur de texte, nous créons le fichier `BoxTest.java` avec le contenu suivant :

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class BoxTest{
    @Test
    public void testBoxCreate() {
        Box b = new Box();
    }
}
```

On sauvegarde le fichier et on compile les tests JUnit :

```
javac -cp .:$J BoxTest.java
```

Est-ce que ça compile?

NON : le test échoue : il n'y a pas d'erreur de syntaxe; l'échec est dû au fait que `Box` n'existe pas encore. On se prépare à commiter le test dans une branche `test-box-create` :

```
$ git branch test-box-create
$ git switch test-box-create
Switched to branch 'test-box-create'
$ git status
On branch test-box-create
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    BoxTest.class
    BoxTest.java

nothing added to commit but untracked files present (use "git add" to track)
```

Question 2 Le fichier `BoxTest.java` n'est pas encore tracké. Nous allons donc devoir l'ajouter à l'index avant de valider les modifications avec un `commit`. Suivez les étapes décrites ci-dessous.

Question 3 Tout d'abord, nous souhaitons éviter de versionner les fichiers `.class`. Comment faire?

Astuce



On peut gérer les `.class` avec un `.gitignore` adapté. Faites-le et committez le `.gitignore` correspondant dans une branche séparée!

Question 4 Committez à présent le test. Attention, nous allons essayer de respecter une convention pour l'écriture des commits. Nous allons indiquer le nom de la feature sur laquelle nous travaillons.

```
$ git add BoxTest.java
$ git commit -m "Biblio Box: ajout testBoxCreate"
[test-box-create c3aed31] Biblio Box: ajout testBoxCreate
1 file changed, 9 insertions(+)
create mode 100644 BoxTest.java
```

Question 5 On a maintenant un test qui échoue. On passe donc à l'implémentation de la fonctionnalité.

Avec notre éditeur, créons le fichier `Box.java` avec le contenu minimal suivant :

```
public class Box {
    public Box(){System.out.println("Box créée");}
}
```

Question 6 Relancez les tests pour vérifier que le test passe à présent.

On recompile les tests :

```
javac -cp .:$J BoxTest.java
```

Cette fois ça compile!

On relance donc les tests pour vérifier que cette fois le test passe :

```
java -jar $J execute --class-path . --scan-class-path
```

Le test passe. Commiter la nouvelle fonctionnalité offerte par le fichier `Box.java`. Toujours depuis la branche `test-box-create`, on la pousse sur le dépôt distant :

```
$ git push -u origin test-box-create
```

L'option `-u` relie la branche locale à la branche distante du même nom. Il suffit de le faire au premier push : ensuite, un simple `git push` depuis cette branche suffit.

On crée ensuite la pull request correspondante sur GitHub, puis on la fusionne (*merge*).

4.1 Faire le ménage après la fusion

Une fois la pull request fusionnée, GitHub propose un bouton *Delete branch* : cliquez dessus pour supprimer la branche distante. Par défaut, GitHub ne la supprime **pas** tout seul.

En local, on revient sur `main`, on la met à jour, puis on supprime la branche de la feature, qui ne sert plus :

```
$ git switch main
$ git pull
$ git branch -d test-box-create
$ git fetch --prune
```

- `git branch -d` refuse de supprimer une branche qui n'a pas été fusionnée : c'est une sécurité. Si vous avez choisi *Squash and merge* sur GitHub, Git ne reconnaît pas la fusion : vérifiez sur GitHub qu'elle a bien eu lieu, puis utilisez `git branch -D`.
- `git fetch --prune` supprime en local la référence `origin/test-box-create` vers la branche distante effacée.

Attention

`git pull` récupère les commits du dépôt distant **et les fusionne dans la branche courante**. Vérifiez donc toujours avec `git status` que vous êtes sur `main` avant de le lancer.

Si vous voulez seulement *voir* ce qui a changé sur le dépôt distant, sans rien modifier dans vos branches, utilisez `git fetch`, puis comparez :

```
$ git fetch
$ git log --oneline main..origin/main
```

`git pull`, c'est en fait `git fetch` suivi d'un `git merge`.



Vous avez réalisé votre première feature. Bravo!

À retenir

Pour chaque nouvelle feature à développer, vous procéderez à présent de la manière suivante :

- Depuis main à jour, créer une branche nommée `test-<nom de la feature>`.
- Basculer sur cette nouvelle branche avec la commande `git switch <nom de la branche>`.
- Implémenter un test qui échoue, et le commiter.
- Implémenter la fonctionnalité.
- Vérifier que le test passe, et commiter.
- Pousser la branche avec `git push`. La première fois : `git push -u origin <branche>`.
- Créer la pull request sur GitHub, la fusionner et supprimer la branche distante.
- Re-basculer sur la branche main et la mettre à jour avec `git pull`.
- Supprimer la branche locale avec `git branch -d <branche>`.



Question 7 On veut pouvoir mettre des trucs dans une Box! Ecrivez d'abord le test correspondant...

Comme pour la feature précédente, on travaille dans une nouvelle branche :

```
$ git switch main
$ git branch test-box-add
$ git switch test-box-add
```

Dans le fichier de tests, `BoxTest.java` on ajoute un nouveau test :

```
import org.junit.jupiter.api.Test;

public class BoxTest {
    @Test
    public void testBoxCreate() {
        Box b = new Box();
    }

    /** on veut pouvoir mettre des trucs dedans */
    @Test
    public void testBoxAdd() {
        Box b = new Box();
        b.add("truc1");
    }
}
```

```
        b.add("truc2");
    }
}
```

On recompile :

```
javac -cp .:$J BoxTest.java
```

Cette compilation échoue comme prévu : il n'y a pas d'erreur de syntaxe ; il manque simplement la nouvelle fonctionnalité. Nous commitons le nouveau test :

```
git commit -a -m "Biblio Box: ajout de testBoxAdd"
[test-box-add 8b10324] Biblio Box: ajout de testBoxAdd
1 file changed, 6 insertions(+)
```

Question 8 Comment implémenter la fonctionnalité demandée par le test ?

Dans le fichier `Box.java`, nous implantons à présent cette fonctionnalité : il faut une méthode `add` pour ajouter un `String` à la boîte. Donc il faut aussi un endroit pour mettre ces `String` : par exemple, un attribut qui serait une liste de `String`. Nous modifions donc la définition de la classe `Box` comme suit :

```
import java.util.ArrayList;
import java.util.List;

public class Box {
    private List<String> contents = new ArrayList<>();

    public Box() {
        System.out.println("Box créée");
    }

    public void add(String truc) {
        this.contents.add(truc);
    }
}
```

On recompile les tests :

```
javac -cp .:$J BoxTest.java
```

Ça compile ! On fait tourner les tests :

```
java -jar $J execute --class-path . --scan-class-path
```

Ça passe! on peut donc commiter la nouvelle fonctionnalité :

```
$ git commit -a -m "Biblio Box: attribut contents et methode add"
[test-box-add 7fef816] Biblio Box: attribut contents et methode add
 1 file changed, 6 insertions(+), 1 deletion(-)
$ git status
On branch test-box-add
nothing to commit, working tree clean
```

Question 9 On voudrait maintenant mettre des Thing dans la boîte. Comment modifier la classe Box pour que cela soit possible? Par exemple, on crée une classe Thing dans un fichier Thing.java :

```
public class Thing {
    private String name;

    public Thing(String name) {
        this.name = name;
    }
}
```

puis on modifie Box.java :

```
import java.util.ArrayList;
import java.util.List;

public class Box {
    private List<Thing> contents = new ArrayList<>();

    public Box() {
        System.out.println("Box créée");
    }

    public void add(Thing truc) {
        this.contents.add(truc);
    }
}
```

Attention

La méthode `add` attend maintenant un `Thing` : le test `testBoxAdd` ne compile plus. Il faut le mettre à jour :



```
@Test
public void testBoxAdd() {
    Box b = new Box();
    b.add(new Thing("truc1"));
    b.add(new Thing("truc2"));
}
```

Testez! et commitez la nouvelle version des classes `Box` et `Thing`.

5 Bibliothèque Box : Éléments suivants du backlog

- on veut pouvoir tester si un truc est dans une boîte. Utiliser la méthode `contains` de l'interface `java.util.List` pour proposer une méthode pour cela dans la classe `Box`. Consultez la documentation de `List` sur <https://docs.oracle.com/en/java/javase/21/docs/api/>.
- on veut pouvoir retirer un truc d'une boîte : après qu'on l'ai retiré, il ne devrait plus être dedans. Si on essaie de retirer un truc qui n'est pas présent dans la boîte, il faudra déclencher/lever une exception.

Une exception est un signal qui se déclenche en cas de problème. Les exceptions permettent de gérer les cas d'erreur et de rétablir une situation stable.

Pour vous aider à utiliser cette nouvelle notion, vous pouvez consulter les liens suivants :

- https://fr.wikibooks.org/wiki/Programmation_Java/Exceptions
- <https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html>

Pour écrire un test qui vérifie qu'une exception est bien levée, JUnit 6 fournit l'assertion `assertThrows` :

```
import static org.junit.jupiter.api.Assertions.assertThrows;
...
@Test
public void divisionWithException() {
    assertThrows(ArithmeticException.class, () -> {
        int i = 1 / 0;
    });
}
```

`assertThrows` prend deux arguments :

- la classe de l'exception attendue : `ArithmeticException.class`;

- le code qui doit la lever, écrit sous la forme d'une **lambda**.

À retenir

Une **lambda** est un morceau de code qu'on passe en argument à une méthode *sans l'exécuter tout de suite*. Elle s'écrit
`() -> { instructions; }`:

- `()` : la lambda ne prend pas de paramètre;
- `->` : sépare les paramètres du code;
- `{ ... }` : le code à exécuter plus tard.



Ici, c'est `assertThrows` qui exécute la lambda, *attrape* l'exception et vérifie qu'elle est du bon type. Si aucune exception n'est levée, ou pas la bonne, le test échoue.

Quand le code tient en une seule instruction, on peut enlever les accolades et le point-virgule :

```
@Test
public void getOnEmptyList() {
    List<String> liste = new ArrayList<>();
    assertThrows(IndexOutOfBoundsException.class, () -> liste.get(0));
}
```

Astuce

`assertThrows` renvoie l'exception levée : on peut donc aussi vérifier son message.



```
ArithmeticException e = assertThrows(
    ArithmeticException.class,
    () -> Math.floorDiv(1, 0));
assertEquals("/ by zero", e.getMessage());
```

Attention



Si vous trouvez sur Internet la syntaxe `@Test(expected = ...)`, c'est du JUnit 4 : elle n'existe plus en JUnit 6.

Consultez éventuellement la section *Assertions* de la documentation officielle de JUnit : <https://docs.junit.org/current/user-guide/>.

- une boîte peut être ouverte ou fermée : on voudrait pouvoir tester avec `b.isOpen()`. On voudrait aussi pouvoir la fermer avec `b.close()`, après quoi elle devrait être fermée, et l'ouvrir avec `b.open()`, après quoi elle devrait être ouverte.
- on veut qu'une boîte permette `b.actionLook()` pour regarder dedans. Cette action retourne une String qui nous dit `"la boîte contient: ceci, cela"` si la boîte est

ouverte et contient les strings "ceci" et "cela". Si la boîte est fermée, elle retourne le String "la boîte est fermée".

Astuce



Si on a une liste `liste` de type `List<String>` et qu'on veut produire la chaîne de ces strings séparées par des virgule-espace on fait :
`String listString = String.join(", ", liste);`
Attention, `String.join` n'accepte que des chaînes : comme la boîte contient des `Thing`, il faut d'abord construire la liste (`List<String>`) de leurs noms.

- on veut pouvoir créer des choses (à mettre dans des boîtes) qui prennent de la place. Une chose de volume 3 devrait s'obtenir par `new Thing(3)`.
- on veut pouvoir connaître la place prise par une chose : `t.volume()`
- on veut pouvoir donner à une boîte une capacité : `b.setCapacity(5)`, et obtenir sa capacité : `b.capacity()`
- quand on n'a pas donné une capacité à une boîte, cette capacité est `-1`.
- on voudrait pouvoir tester si il reste de la place dans une boîte pour une nouvelle chose : `b.hasRoomFor(t)`. Quand on ajoute une chose à une boîte, elle consomme son volume de la capacité de la boîte ; et il en reste d'autant moins pour les choses suivantes.
- Une boîte de capacité `-1` est considérée comme ayant une capacité illimitée.
- on voudrait qu'une boîte permette `b.actionAdd(t)` qui permette de mettre une chose dans une boîte mais seulement s'il y a suffisamment de place. Quand l'action échoue, elle lève une exception.
- `b.actionAdd(t)` devrait aussi échouer si la boîte est fermée.
- il faudrait pouvoir donner un nom à chaque chose pour qu'elle s'affiche de manière intelligible (elle affiche juste son nom) : `t.setName("bidule")`
On convertit un objet en string avec sa méthode `t.toString()`, que Java appelle automatiquement dans `System.out.println(t)` ou dans `"chose: " + t`. Par défaut, elle affiche quelque chose comme `Thing@1b6d3586` : il faut donc la redéfinir dans la classe `Thing`, avec la signature `public String toString()`.
- on voudrait pouvoir tester si une chose a un nom donné : `t.hasName("bidule")`
- on voudrait qu'une boîte permette `b.find("bidule")` pour trouver et retourner l'objet ayant ce nom et contenu dans la boîte. La méthode lève une exception si elle n'a rien trouvé.
- on voudrait que `find` lève aussi une exception si la boîte est fermée.
- on voudrait pouvoir donner à la fois le volume et le nom d'une chose dans son constructeur : `new Thing(3, "bidule")` (au lieu d'avoir à utiliser `setName` ensuite).
Pour cela il faut ajouter à la classe `Thing` un constructeur qui a deux arguments, un `int` et un `String`.

Pour ne pas dupliquer de code, les autres constructeurs peuvent appeler ce nouveau constructeur avec `this(...)`. Par exemple, le constructeur `Thing(int volume)` peut se contenter d'appeler `this(volume, "")`.

- on voudrait de même pouvoir optionnellement préciser dans le constructeur si une boîte est ouverte ou fermée, et sa capacité :

6 JSON

À retenir



JSON est une notation simplifiée pour représenter des structures de données et échanger des données entre applications, notamment sur le web.

Voir <http://en.wikipedia.org/wiki/JSON> et <https://www.json.org/json-fr.html>.

Voici un exemple de JSON qui représente une liste de boîtes :

```
[
  {
    "name": "Boite 1",
    "capacity": 3,
    "open": true
  },
  {
    "name": "Boite 2",
    "capacity": 6,
    "open": false
  },
  {
    "name": "Boite 3",
    "capacity": -1,
    "open": true
  }
]
```

6.1 Méthode statique en java

On peut ajouter à la classe `Box` une méthode statique `Box.fromJSON()` qui permet de créer une liste de boîtes à partir d'un objet JSON du type ci-dessus. On pourra par exemple utiliser en java la librairie pour Gson parser du JSON; son jar est disponible sur Célène. Voir la documentation officielle sur <https://www.javadoc.io/doc/com.google.code.gson/gson/latest/com.google.gson/module-summary.html>, et quelques exemples dans le manuel utilisateur <https://github.com/google/gson/blob/master/UserGuide.md#object-examples>.

Pour utiliser la classe Gson, il faut inclure le jar de gson dans le *classpath*, et importer la classe Gson avec `import com.google.gson.Gson;`.

Pour ouvrir un fichier en lecture en java et l'utiliser dans gson, il faut :

- i Créer une instance de FileReader avec :

```
FileReader fr = new FileReader("chemin/du/fichier.json");
```

- ii Passer l'objet fr à la méthode fromJson d'un objet de la classe Gson. Comme le fichier contient une *liste* de boîtes, on demande un tableau de Box :

```
Gson gson = new Gson();
```

```
Box[] boites = gson.fromJson(fr, Box[].class);
```

6.2 Backlog utilisant JSON

- on voudrait pouvoir construire une liste de choses à partir d'une description en JSON. Il faudrait pour cela une méthode statique `Thing.fromJson(data)` construisant une chose à partir des données la concernant.
- on voudrait pouvoir construire une liste de boîtes et de choses à partir d'une description en JSON. Pour cela, il faudrait que chaque élément de la liste JSON ait une clé `type` ayant pour valeur soit `box` soit `thing`. Il faudrait aussi une méthode statique `listFromJSON()` qui prenne en argument une liste de données obtenue de JSON et pour chacune construise soit une boîte soit une chose selon la valeur de la clé `type`.
- on voudrait pouvoir faire une sauvegarde JSON complète d'une liste de boîtes et de leur contenu.